

Lispy | 

(Task 1 of 11) In this tutorial, we will learn about **variable assignments** and **mutable variables**.

The following program illustrates the new concepts.

Lispy [Run]

JavaScript

```
(defvar x 2)      let x = 2;
x                console.log(x);
(set! x (+ x 1))  x = x + 1;
x                console.log(x);
(set! x (* x 2))  x = x * 2;
x                console.log(x);
```

This program produces 2 3 6. It first defines `x` and binds `x` to 2. The first variable assignment (`set! x (+ x 1)`) **mutates** (the binding of) `x`. After that, `x` is bound to the value of `(+ x 1)`, which is 3. The next variable assignment again mutates `x` and binds `x` to the value of `(* x 2)`; this uses the new value of `x`, which is 3, resulting in `(* 3 2)`, which is 6.

Lispy | 

(Task 2 of 11) What is the result of running this program?

Lispy [Run]

Pseudo

```
(defvar rent 10)      let rent = 10
(set! rent (* 10 2))  rent = 10 * 2
rent                  print(rent)
```

20

2

You predicted the output correctly 🎉🎉🎉

3

`rent` is bound to 10 initially. The variable assignment mutates `rent` to 20. So, when we print the value of `rent` after the variable assignment, we see 20.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 3 of 11) What is the result of running this program?

Lispy [Run]

Pseudo

```
(defvar m 40)      let m = 40
(defvar n m)        let n = m
(set! n 22)         n = 22
m                   print(m)
n                   print(n)
```

40 22

5

You predicted the output correctly 🎉🎉🎉

6

`m` and `n` are bound to `40`. The variable assignment mutates the binding of `n`. So, `n` is eventually bound to `22`.

Click [here](#) to run this program in the Stacker.

Lispy |

(Task 4 of 11) What is the result of running this program?

Lispy [Run]

JavaScript

```
(defvar x (mvec 55))      let x = [ 55 ];
(defvar v (mvec x 55 55)) let v = [ x, 55, 55 ];
(set! x (mvec 66))        x = [ 66 ];
v                          console.log(v);
```

`#(#(55) 55 55)`

8

You predicted the output correctly 🎉🎉🎉

9

`x` is first bound to a one-element vector. `v` is bound to a three-element vector, of which the first element is the one-element vector. `(set! x (mvec 66))` binds `x` to a new vector. This doesn't impact `v` because the `0`-th element of `v` is still the one-element vector.

Click [here](#) to run this program in the Stacker.

Lispy |

(Task 5 of 11) What is the result of running this program?

Lispy [Run]

Python

```
(defvar x 80)  x = 80
(defvar y x)   y = x
(set! x 32)    x = 32
x              print(x)
y              print(y)
```

`32 80`

11

You predicted the output correctly 🎉🎉🎉

12

The first definition binds `x` to `80`. The second definition binds `y` to the value of `x`, which is `80`. The variable assignment mutates the binding of `x`, so `x` is now bound to `32`. But `y` is still bound to `80`.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 6 of 11) What is the result of running this program?

Lispy [Run 

JavaScript

```
(defvar x 59)      let x = 59;
(defvar v (mvec 68 57 x)) let v = [ 68, 57, x ];
(set! x 74)        x = 74;
v                  console.log(v);
```

#(68 57 59)

14

You predicted the output correctly 🎉🎉🎉

15

`v` is bound to the vector created by `(mvec 68 57 x)`. `(mvec 68 57 x)` is a function call. `x` is evaluated at the point of evaluating `(mvec 68 57 x)`. That is why the vector's content is 68, 57, and 59. The subsequent variable assignment mutates `x`. But this doesn't impact the vector because the vector refers the value 59 rather than `x`.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 7 of 11) What is the result of running this program?

Lispy [Run 

Python

```
(defvar x 12)      x = 12
(deffun (f y)      def f(y):
  (set! y 0)        y = 0
  x)                return x
(f x)               print(f(x))
```

⚠️ Scala 3 behaves differently.

12

17

You predicted the output correctly 🎉🎉🎉

18

The function call binds `y` to 12. The variable assignment mutates the value of `y` to 0, but `x` is still bound to 12.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 8 of 11) What is the result of running this program?

Lispy [Run ▶] <pre>(defvar x 12) (deffun (f y) (set! x 0) y) (f x)</pre> Pseudo <pre>let x = 12 fun f(y): x = 0 return y end print(f(x))</pre>	
12	20
You predicted the output correctly 🎉🎉🎉 The function call binds <code>y</code> to 12. The variable assignment mutates the value of <code>x</code> to 0, but <code>y</code> is still bound to 12. Click here to run this program in the Stacker.	21
(Task 9 of 11) What did you learn about variable assignment from these programs?	22
assigns to a value	23
(Task 10 of 11) Variable assignments change <i>only</i> the mutated variables. That is, variables²⁴ are not aliased. (Note: some programming languages (e.g., C++ and Rust) allow variables to be aliased. However, even in those languages, variables are not aliased by default.)	24
Any feedback regarding these statements? Feel free to skip this question.	25
(You skipped the question.)	26
(Task 11 of 11) Please scroll back and select 1-3 programs that make the above point. You don't need to select <i>all</i> such programs.	27
(You selected 2 programs)	28
Okay. How do these programs (16,19) support the point?	29

Show non-mutation

30

Let's review what we have learned in this tutorial.

31

Variable assignments change *only* the mutated variables. That is, variables are not aliased.

(Note: some programming languages (e.g., C++ and Rust) allow variables to be aliased. However, even in those languages, variables are not aliased by default.)

You have finished this tutorial 🎉🎉🎉

Please [print](#) the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781412748163